A Project Synopsis Report

On

AI Web Scraping & Intelligent Content Summarizer

In partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

Submitted by

Gaurav Chauhan (1230432288)

Gaurav Yadav (1230432292)

Golu Rai (1230432295)

Deepal Yadav (1230432259)



Under the supervision of

Dr. Harsh Dev (Head of Department)

Department of Computer Science & Engineering

to the

School of Engineering

BABU BANARASI DAS UNIVERSITY, LUCKNOW

1. Introduction

Almost every activity performed online today begins with an act of agreement. Installing an application, subscribing to a service, renting a flat, opening a bank account or accepting an internship requires a person to accept a written document. Terms of service, privacy policies, rental agreements, offer letters and service contracts are usually long, written in dense semi-legal language, and structured so that the most consequential sentences are difficult to notice. The overwhelming majority of users accept such documents without reading them, not out of carelessness, but because the cost of reading is unreasonably high compared with the time available.

The consequence of this reading gap is not merely theoretical. A clause that quietly enables automatic renewal, a cancellation window that closes thirty days before the billing date, a fee triggered only under specific conditions, or a permission allowing personal data to be shared with third parties can have a real financial and privacy impact. The information is present in the document, but it is effectively invisible because it is buried inside pages of uniform text. At the same time, the content a student or professional needs is spread across documents, web pages and video lectures, and there is no convenient environment in which all of it can be ingested, understood, searched by meaning and questioned in natural language.

Recent progress in Natural Language Processing has made a solution feasible. Transformer architectures introduced self-attention, allowing a word to be represented in the context of the entire sentence. Pretrained encoders such as the BERT family can be fine-tuned on a modest labelled dataset and still achieve strong performance on specialised classification tasks. Sentence-level embedding models represent a passage as a dense vector, so that two passages expressing the same idea in different words lie close together in vector space. Retrieval-Augmented Generation (RAG) then combines this semantic retrieval with a generative model, so that answers are produced from retrieved evidence rather than from the model's memory alone.

The proposed project brings these capabilities together into a single web-based platform. The system accepts content from five kinds of sources — PDF, DOCX and TXT files, web page URLs and YouTube transcripts — and converts them into a common internal representation in which page numbers and section names are preserved. The content is segmented into sections and clauses, and each clause is passed through a supervised machine learning model trained specifically for this project to predict its category, such as financial, automatic renewal, cancellation, privacy, liability, arbitration or termination.

This trained classifier is the academic core of the work. It is not a general-purpose language model consumed through an API it is a pretrained transformer encoder fine-tuned by the project team on a labelled clause dataset that the team compiles, cleans, validates and splits, and whose performance is measured using standard supervised metrics. Around this model the system adds a semantic search layer built on sentence embeddings and a FAISS index, a retrieval-augmented question answering layer, an attention-scoring component that highlights clauses deserving closer reading, multi-level summarization, and a comparison module that reports what changed between two versions of an agreement. The platform is an informational and comprehension aid; it does not give legal advice, make legal determinations or act on the user's behalf.

2. Problem Statement

Users are routinely required to accept documents whose contents they do not understand, and existing tools do not solve this in a way that is both accurate and verifiable. A typical terms of service document runs to several thousand words, yet the clauses carrying the greatest consequence, such as an automatic renewal with a thirty day notice requirement, occupy only a few lines. The difficulty is not the volume of text but the uneven distribution of what actually matters within it.

2.1 What the Project Solves

The project develops an integrated platform that ingests content from heterogeneous sources, segments it into meaningful units, classifies each unit into a clause category using a machine learning model trained for this purpose, assigns an attention level indicating how closely the unit deserves to be read, answers natural-language questions using retrieval-augmented generation with source references, and compares two versions of a document to report substantive changes.

2.2 Gaps in Existing Systems

1. Keyword dependence. Conventional readers match literal strings. A user searching for "cancel" will not find a clause phrased as "terminate this agreement", even though it is the clause they need.
2. No classification by obligation type. Existing tools treat a document as an undifferentiated block of text and do not distinguish a payment obligation from a privacy permission or an arbitration requirement.

3. Generic summarizers lose critical detail, preserving the general theme while discarding exactly the numbers, deadlines and conditions that matter most, and general assistants produce fluent answers that are not traceable to any passage, so the user cannot verify them.
4. Single-source design. A tool that reads PDFs does not read web pages; a tool that summarizes videos does not read contracts. Related content cannot be analyzed together.
5. No version comparison at the level of meaning. Character-level difference tools produce noise when what the user needs is a statement of which obligations changed.
6. Absence of an evaluated, task-specific model. Most document assistants are thin wrappers over a hosted language model, with no trained component whose accuracy has been measured on a held-out test set.

2.3 Need for the Proposed Work

What is missing is a single system that understands the structure of a document rather than only its words, classifies clauses into meaningful categories using a model trained for that purpose, retrieves passages semantically rather than by keyword, grounds every explanation in a traceable reference, and does all of this across several content formats.

Building such a system requires a supervised learning component. A rule based or purely prompt based approach cannot reliably distinguish a termination clause from a cancellation clause, or an indemnification clause from a general liability statement, in a consistent and measurable way. A classifier fine tuned on labelled examples can be evaluated quantitatively, improved with additional data, and produces a confidence value the rest of the system can use. This is the gap the project sets out to fill.

3. Objectives

The objectives are ordered so that the machine learning objectives, which constitute the principal contribution, are stated first.

1. **To prepare a labelled clause dataset.** To collect, clean, segment and manually label document clauses against defined categories, validating consistency before training.
2. **To train a supervised transformer based clause classifier.** To fine tune a pretrained encoder from the BERT family so that it assigns a category and confidence score to unseen clauses.

3. **To evaluate the trained model rigorously.** To measure accuracy, precision, recall, F1-score, macro and weighted F1 and the confusion matrix, and analyze class wise behaviour.
4. **To design a multi-source ingestion pipeline.** To extract clean text and metadata from PDF, DOCX and TXT files, web pages and YouTube transcripts.
5. **To build an NLP preprocessing and clause segmentation pipeline.** To develop an NLP preprocessing and clause segmentation pipeline that cleans and normalizes extracted content, performs sentence segmentation and section identification, and prepares meaningful textual units while preserving relevant source metadata for further AI/ML analysis.
6. **To implement semantic search.** To embed chunks with a Sentence Transformer and index them in FAISS so passages are retrieved by meaning rather than wording.
7. **To develop a Retrieval-Augmented Generation module.** To embed the question, retrieve and rerank relevant chunks, and obtain a grounded answer carrying source references.
8. **To design an attention and importance analysis mechanism.** To combine predicted category, model confidence and content signals into an indicator marking clauses Low, Medium, High or Very High in reading priority.
9. **To implement multi-level summarization.** To produce overall, section wise, important clause and question focused summaries hierarchically.
10. **To provide document comparison.** To compare two versions and report added, removed and modified clauses, noting changes in fees, renewal, cancellation and data use.
11. **To develop a secure and usable web application.** To deliver a React interface on a FastAPI service with authenticated accounts, isolated storage and an analysis dashboard.
12. **To test and document the system.** To validate modules and the integrated pipeline through testing, and document the design, training methodology and results.

4. Scope of the Project

4.1 Included in the scope

The project covers ingestion from PDF, DOCX and TXT files, public web pages and YouTube videos exposing a transcript, with extraction of page numbers, section headings and timestamps; NLP preprocessing consisting of cleaning, normalization, sentence segmentation, section identification and clause segmentation; preparation and labelling of a clause dataset with deduplication, label validation and a documented train, validation and test split; fine tuning and full evaluation of a pretrained transformer encoder for multi-class clause classification; semantic embedding of chunks with FAISS indexing and a RAG pipeline that answers questions from retrieved context with source references; identification of clauses

meriting closer attention on a four level scale; overall, section wise, clause focused and question focused summarization; comparison of two versions of a document; and a web dashboard with authentication, document history, clause tables, filters and charts.

4.2 Not Included in the Scope

The project does not train a large language model from scratch, since it fine tunes a pretrained encoder and uses an existing generative model only for explanation and question answering. It performs no legal decision making, makes no determination of whether a clause is enforceable, lawful or fair in any jurisdiction, and offers no guarantee of legal validity. It is an aid to understanding rather than a replacement for professional legal advice, and the interface states this. It does not execute legal or financial actions such as cancelling a subscription on the user behalf. Languages other than English, and optical character recognition for scanned documents, are identified as future extensions rather than first version features.

4.3 Assumptions and Constraints

The project assumes that uploaded documents contain machine readable text, that web pages permit automated extraction of their main content, and that submitted videos expose a transcript. It further assumes that a sufficiently diverse labelled dataset can be assembled within the timeline, since classifier quality depends on this. The main constraints are the computational resources available for fine tuning, managed by using a base sized transformer and free tier GPU environments, and the latency and usage limits of any hosted language model.

5. Literature Review

5.1 Transformer Architectures and Transfer Learning

The transformer architecture introduced by Vaswani and colleagues in 2017 replaced recurrent processing with self attention. Devlin and colleagues then proposed BERT, a bidirectional encoder pretrained on masked language modelling, and showed that such an encoder could be fine tuned on a comparatively small labelled dataset to achieve strong downstream classification results. Later variants including RoBERTa, DistilBERT and domain adapted models such as LEGAL-BERT confirmed that the choice of checkpoint measurably affects

accuracy. This supports the methodology adopted here, in which a pretrained encoder is fine tuned on a task specific clause dataset rather than trained from random initialization.

5.2 Legal and Contractual Text Analysis

The CUAD dataset released by Hendrycks and colleagues provides expert annotated commercial contracts with many labelled clause spans and has become a common benchmark. Earlier work by Lippi and colleagues on CLAUDETTE applied machine learning to detect potentially unfair clauses in online terms of service, while ContractNLI framed contract understanding as a natural language inference problem. This literature establishes clause level classification as a well posed and tractable problem. Its limitation is that these systems are research prototypes or enterprise tools, restricted largely to formal contracts, with no interactive interface through which an ordinary reader can question their own document.

5.3 Dense Retrieval and Retrieval-Augmented Generation

Reimers and Gurevych proposed Sentence-BERT, which modifies a pretrained transformer using a siamese structure so that semantically similar sentences produce embeddings close under cosine similarity, making semantic search practical at scale. Karpukhin and colleagues showed with Dense Passage Retrieval that dense retrieval can outperform sparse methods such as BM25, and Johnson and colleagues developed FAISS for efficient similarity search over dense vectors. Lewis and colleagues then introduced Retrieval-Augmented Generation, in which a retriever selects relevant passages and a generative model conditions its output on them, so that generated text is grounded in retrievable evidence whose source can be cited. The present project adopts this architecture and requires that every answer carry a page, section or timestamp reference.

5.4 Automatic Summarization

Summarization research distinguishes extractive approaches from abstractive ones. Models such as BART and PEGASUS demonstrated strong abstractive performance but are constrained by a fixed input length, a problem for multi-page documents. Hierarchical summarization, in which chunks are summarized individually before being combined, is the established method for inputs exceeding the context window, and this project adopts it.

5.5 Identified Gaps

The individual components are well supported by existing research. What has not been widely combined is a system unifying a purpose trained clause classifier, semantic retrieval, grounded

generation, attention analysis and version comparison behind one interface that also accepts informal sources such as web pages and video transcripts. The proposed work sits in that gap. It does not claim to advance transformer architecture; its contribution lies in constructing and evaluating a task specific labelled dataset and classifier, and integrating that classifier into a complete and verifiable pipeline.

6. Methodology / Proposed System

The system is organized as a layered pipeline. Content enters through an ingestion layer, passes through NLP preprocessing and clause segmentation, is analyzed by the machine learning layer, and is served to the user through a retrieval and generation layer. This section presents the architecture and then describes each stage in the order data flows through it, with the machine learning stages described in the greatest detail.

6.1 System Architecture

The proposed AI Web Scraping & Intelligent Content Summarizer consists of several interconnected modules. The user interacts with the system through a React-based web application, where documents or online sources such as PDF, DOCX, TXT, web pages, and YouTube content can be provided.

The Data Ingestion and Content Extraction modules collect and extract relevant text along with page and section information. The extracted content is cleaned and segmented through NLP preprocessing.

The processed content is then passed to the AI/ML Pipeline, which performs transformer-based embedding, clause classification, entity extraction, and importance detection. A RAG Pipeline retrieves relevant information using query embeddings and FAISS-based semantic search.

The retrieved context is provided to the Large Language Model (LLM) to generate a meaningful summary. Finally, the Dashboard presents the summary, important information, analysis, and source/page/section references to the user.

User → Web Application → Data Ingestion → Content Extraction → NLP → AI/ML → RAG → LLM → Final Analysis → Dashboard

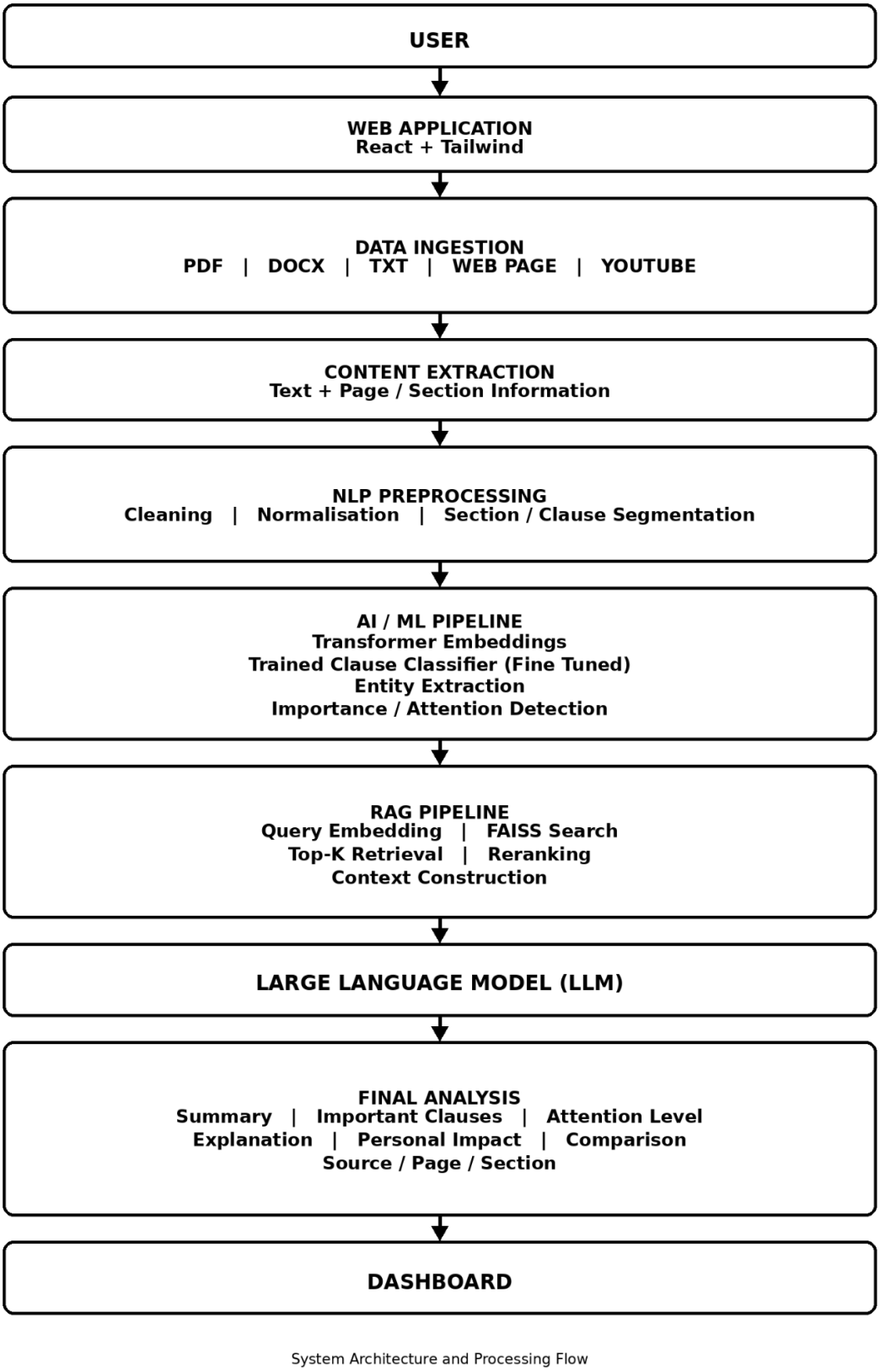


Fig. 6.1 Block diagram of the proposed platform

6.2 Tools and Technologies

The proposed system will use a practical combination of web development, machine learning, natural language processing, document extraction, semantic retrieval and language generation technologies. The final selection may be adjusted during implementation based on compatibility and available computing resources.

1. **Frontend Technologies**

   React.js will be used to develop the interactive user interface, while Tailwind CSS will support responsive page styling. Recharts may be used to visualize clause categories and attention levels on the dashboard.

2. **Backend Technologies**

   Python will be the primary backend programming language. FastAPI will provide REST APIs for document upload, analysis, summarization, question answering and comparison. SQLAlchemy will be used for database access.

3. **Machine Learning and NLP**

   The main machine-learning component will be a supervised transformer-based clause classifier. A suitable pretrained transformer, such as a BERT-family model, will be fine-tuned on a labeled clause dataset using PyTorch and Hugging Face Transformers. spaCy or NLTK may support text-processing tasks. Pandas and NumPy will assist with dataset preparation, while scikit-learn will calculate evaluation metrics such as accuracy, precision, recall and F1-score.

4. **Document and Content Extraction**

   PyMuPDF will extract text and page information from PDF files. python-docx will process DOCX documents, and Python text-processing facilities will handle TXT files. Trafilatura or BeautifulSoup may extract main text from webpages. YouTube transcript text will be obtained through an appropriate transcript API or available transcript source.

5. **Embeddings, Vector Search and RAG**

Sentence Transformers will generate semantic embeddings for document chunks and user questions. FAISS will index vectors and retrieve semantically relevant passages. A Retrieval-Augmented Generation (RAG) pipeline will combine retrieved passages with the user's question and pass the context to a configurable large language model (LLM), such as an available hosted API or local model. The LLM supports grounded question answering and explanations; it does not replace the trained clause classifier.

6. **Database and Security**

PostgreSQL will store user accounts, document metadata, extracted clauses, analysis results and conversation history. JWT-based authentication and password hashing with bcrypt or Argon2 will protect account access. API keys will be stored in environment variables, and uploaded files will be validated.

7. **Development, Testing and Deployment**

Git and GitHub will be used for version control, and Visual Studio Code may be used as the development environment. Postman will support API testing, while Pytest will be used for automated tests. The frontend may be deployed on Vercel and the backend on Render, Railway or a similar hosting service, subject to available resources.

Layer	Technologies
Frontend	React.js, Tailwind CSS, Recharts
Backend	Python, FastAPI
Machine Learning	PyTorch, Hugging Face Transformers, Sentence Transformers, scikit-learn
NLP and data handling	spaCy, NLTK, Pandas, NumPy
Document processing	PyMuPDF, python-docx, Trafilatura, BeautifulSoup, YouTube Transcript API
Vector search	FAISS
RAG orchestration	Custom RAG pipeline, with LangChain where it simplifies implementation
Language model	Configurable provider such as OpenAI, Gemini, Groq or a local model
Database	PostgreSQL with SQLAlchemy

Layer	Technologies
Authentication	JWT for session tokens, bcrypt or Argon2 for password hashing
Development and deployment	Git, GitHub, VS Code, Postman, Pytest; Vercel and Render or Railway

The language model provider is kept configurable behind a single backend interface, so the project does not depend on one vendor and a local model can be substituted if required.

6.3 Database Design

A relational database is used because the data is strongly structured: documents belong to users, chunks and clauses to documents, messages to conversations. PostgreSQL is proposed, accessed through SQLAlchemy.

Entity	Purpose and key attributes
Users	Account identity, email, hashed password, registration timestamp
Documents	Owner, title, source type, original reference, checksum, processing status
Chunks	Parent document, chunk text, page and section metadata, embedding reference
Clauses	Parent document, text, predicted category, confidence, attention level, location
Conversations	A question answering session associated with a document
Messages	Questions and answers within a conversation, with the references used
Model Versions	Identifier, training date, dataset version and evaluation metrics of each model

The Model Versions table is included deliberately, because the classifier will be retrained as the dataset grows and it is necessary to record which version produced a stored prediction. Embeddings themselves are held in the FAISS index, with the database storing the linking identifiers.

6.4 Backend API Design

The backend exposes a REST API implemented with FastAPI. The endpoints below are representative.

```
POST   /api/auth/register                POST   /api/auth/login
POST   /api/documents/upload             POST   /api/documents/url
POST   /api/documents/youtube            GET    /api/documents
GET    /api/documents/{id}               POST   /api/documents/{id}/ask
POST   /api/documents/{id}/summarize     GET    /api/documents/{id}/clauses
```

```
GET    /api/documents/{id}/risk-analysis
POST   /api/documents/compare
```

6.5 Security and Privacy Considerations

The platform handles documents that are frequently personal or confidential, so security is treated as a design requirement rather than an afterthought.

Users authenticate with an email and password, with sessions maintained through signed JSON Web Tokens of limited expiry and passwords hashed using bcrypt or Argon2 with a per-user salt. Uploads are validated by extension and content type, size limited, and stored under generated names so user supplied filenames cannot influence the storage path. Every database query is filtered by the authenticated user identifier, so documents cannot be reached by guessing an identifier. Provider keys are held in server side environment variables and never exposed to the frontend. Request bodies are validated with Pydantic schemas and submitted URLs checked to prevent requests reaching internal network addresses. Clients receive generic error messages while diagnostics are recorded server side, and logs never hold document content, tokens or personal data. Users can delete a document, which removes its text, clauses and vectors, and the interface states that content is transmitted to the configured language model provider during question answering.

7. Module Description

The system is divided into seventeen modules following the flow of data through the platform, which keeps their interfaces narrow and allows independent development and testing.

Module	Description
1. User Authentication	Registration, login, password hashing, token issue and verification, and ownership checks restricting every request to the authenticated user documents.
2. Multi-Source Ingestion	Accepts an uploaded file, a web page URL or a video link, validates it, identifies the source type and queues it for processing.
3. Document Extraction	Applies the appropriate extractor to each source type, producing raw text with page numbers, section headings and timestamps.
4. NLP Preprocessing	Processes the extracted content from PDF, DOCX, TXT, web pages and YouTube transcripts to convert raw text into a clean and structured form. The module performs text cleaning, normalization, sentence segmentation and section identification, while removing unnecessary formatting and inconsistencies. It preserves important document information such as headings, page numbers and timestamps for later reference. The processed text is then organized into meaningful textual

Module	Description
	units and passed to the clause segmentation, classification, semantic embedding and summarization stages.
5. Clause Segmentation	Identifies section boundaries and groups sentences into clauses representing self contained obligations, carrying forward citation metadata.
6. Dataset Management	Import of candidate clauses, the labelling interface, deduplication, label validation, dataset versioning and the stratified splits.
7. ML Model Training	The fine tuning pipeline: tokenization, model construction, training loop, class weighted loss, validation, early stopping, checkpoint selection and saving. Runs offline.
8. Clause Classification	Loads the trained model in the backend and classifies incoming clauses in batches, returning a category and confidence score.
9. Embedding Generation	Splits documents into overlapping, boundary aware chunks and encodes each into a dense vector using the selected Sentence Transformer.
10. Semantic Search	Builds and maintains the FAISS index, performs similarity search and maps results back to source chunks and metadata.
11. RAG Question Answering	Query embedding, retrieval, optional reranking, context construction and the grounded answer with source references.
12. Attention Analysis	Computes the attention score from classifier output, extracted entities and content signals, assigning the four level indicator with an explanation.
13. Summarization	Overall, section wise, important clause and question focused summaries using the hierarchical strategy.
14. Document Comparison	Aligns clauses across two versions and reports additions, removals and modifications, emphasising fees, renewal, cancellation and privacy changes.
15. Dashboard and Visualization	Clause table with filters, category and attention distributions rendered with Recharts, the question answering interface and comparison view.
16. Database and History	Persists users, documents, chunks, clauses, conversations, messages and model versions, and provides document history and re-access.
17. Testing and Evaluation	Unit tests for extraction, preprocessing and scoring, integration tests for the pipeline, and the scripts producing the metrics and confusion matrix.

8. Expected Outcomes

The expected deliverables are stated below. No numerical performance figures are claimed at this stage, since the model has not yet been trained; measured values will be reported in the final project report.

1. **A trained clause classification model.** A classifier fine tuned on a labelled dataset prepared during this project, assigning a category and confidence score to unseen clauses.

2. **Semantic retrieval and RAG question answering.** A vector search facility locating passages by meaning rather than keyword, and an interface in which the user asks questions in ordinary language and receives answers built from their own document.
3. **Identification of clauses requiring attention.** A four level attention indicator accompanied by an explanation of the factors that produced it.
4. **Plain-English explanations with verifiable references.** Clause level explanations an ordinary reader can follow, each linked to its page, section or timestamp for verification.
5. **Multi-source processing and comparison.** Ingestion of PDF, DOCX, TXT, web and video sources through one pipeline, and a comparison view reporting added, removed and modified clauses.

9. Project Limitations

The following limitations are recognised at the outset, since a system that overstates its reliability could lead a user to place more confidence in its output than is warranted.

1. Classifier performance depends on the quality, size and diversity of the labelled dataset, so clauses from document types or writing styles absent from training data will be classified less accurately.
2. The category set cannot cover every clause in every kind of document. Clauses fitting no defined category fall into Other, and specialised domains may need categories not included here.
3. Scanned documents contain no extractable text layer and would require optical character recognition, which is outside the core deliverable; such files are detected and reported rather than processed incorrectly.
4. Pages rendering content through client side scripting, pages restricting automated access and videos without a transcript cannot be processed reliably.
5. Generated explanations may still contain errors despite being grounded in retrieved content, which is why source references accompany every output.
6. The attention indicator is a reading priority signal derived from interpretable factors, not a legal risk assessment, and the system as a whole is an informational tool that does not constitute legal advice or replace a qualified professional.
7. Externally hosted language model services introduce latency, cost and usage limits outside the control of the project.
8. The first version supports English only; other languages would require a multilingual encoder and a correspondingly labelled dataset. Document comparison relies on semantic alignment and cannot be guaranteed complete, so both versions are always shown for verification.

10. Hardware and Software Requirements

Hardware	Specification
Development machine	Intel Core i5 or equivalent, 8 GB RAM minimum with 16 GB recommended, 100 GB free disk space
Model fine-tuning	CUDA capable GPU with at least 8 GB memory, or a cloud GPU environment such as Google Colab or Kaggle
Deployment	Cloud instance with 2 vCPU and 4 GB RAM as a minimum for CPU based inference, with a stable internet connection

Software	Details
Operating system	Windows 10 or 11, Ubuntu 20.04 or later, or macOS
Languages	Python 3.10 or later, JavaScript (ES6+)
ML and NLP libraries	PyTorch, Hugging Face Transformers, Sentence Transformers, scikit-learn, spaCy, NLTK, Pandas, NumPy
Vector search and documents	FAISS; PyMuPDF, python-docx, Trafilatura, BeautifulSoup, YouTube Transcript API
Backend and frontend	FastAPI with Uvicorn; React.js with Tailwind CSS and Recharts, built using Node.js and npm
Database	PostgreSQL with SQLAlchemy and Alembic
Development and deployment	VS Code, Git and GitHub, Postman, Jupyter Notebook, Pytest; Vercel and Render or Railway

11.Work Plan / Timeline

The project is planned over eight months. Dataset preparation and model training are placed early because they carry the greatest uncertainty and every downstream module depends on them, and some phases overlap so frontend development can proceed while training experiments run.

Month	Planned Activities	Deliverable
Month 1	Literature review, requirement analysis, finalisation of the category set, evaluation of extraction libraries, project setup	Requirement specification and annotation guideline
Month 2	Multi-source ingestion and extraction layer, NLP preprocessing, clause segmentation, metadata handling	Ingestion and segmentation pipeline
Month 3	Dataset collection, cleaning, clause extraction, manual labelling, deduplication, validation and stratified splitting	Labelled and validated clause dataset
Month 4	Fine tuning experiments across candidate checkpoints, hyperparameter tuning, validation, early stopping, model selection	Trained clause classification model

Month	Planned Activities	Deliverable
Month 5	Full evaluation, confusion matrix and class-wise analysis, error analysis, integration of the classifier into the backend	Evaluation report and deployed classifier
Month 6	Embedding generation, FAISS indexing, retrieval tuning, RAG pipeline, prompt design, source referencing, Summarization	Semantic search and question answering
Month 7	Attention scoring and weight calibration, document comparison, React dashboard, Visualizations, authentication and database integration	Integrated web application
Month 8	Unit and integration testing, performance tuning, security review, deployment, final documentation and presentation	Deployed system and final report

12. References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin, "Attention is all you need," in Proc. NeurIPS, Long Beach, CA, USA, 2017, pp. 5998-6008.

[2] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, Minneapolis, MN, USA, 2019, pp. 4171-4186.

[3] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv preprint arXiv:1907.11692, 2019.

[4] V. Sanh, L. Debut, J. Chaumond and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv preprint arXiv:1910.01108, 2019.

[5] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras and I. Androutsopoulos, "LEGAL-BERT: The muppets straight out of law school," in Findings of the ACL: EMNLP 2020, 2020, pp. 2898-2904.

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in Proc. EMNLP-IJCNLP, Hong Kong, 2019, pp. 3982-3992.

[7] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in Proc. EMNLP, 2020, pp. 6769-6781.

[8] J. Johnson, M. Douze and H. Jegou, "Billion-scale similarity search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535-547, 2021.

[9] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020, pp. 9459-9474.

[10] D. Hendrycks, C. Burns, A. Chen and S. Ball, "CUAD: An expert-annotated NLP dataset for legal contract review," in Proc. NeurIPS Datasets and Benchmarks Track, 2021.

[11] M. Lippi et al., "CLAUDETTE: an automated detector of potentially unfair clauses in online terms of service," Artificial Intelligence and Law, vol. 27, no. 2, pp. 117-139, 2019.